Parameterised Ones Counter Using Verilog HDL ONES COUNTER

Project overview

This project implements a parameterised Ones Counter, also known as a Population Counter. It counts the number of logic-1 bits in an N-bit input and returns that count as a binary number.
Unlike a fixed-width design, the input and output widths are controlled by parameters. The same RTL can therefore be reused for different input sizes by changing the parameter values.

Input width	Output width	Design style	Logic type
DATA_WIDTH (default 10)	COUNT_WIDTH = $clog2(DATA_WIDTH+1)	Behavioral RTL	Combinational

What is an Ones Counter?

An Ones Counter examines every bit of a binary input and adds the bit values. A bit with value 1 contributes one to the count; a bit with value 0 contributes zero.

> **Example: For input `d = 10'b0000010111`, there are five logic-1 bits. The output is `y = 4'b0101`, which is decimal 5.**

Understanding the parameters

Parameter	Purpose	Default / example
DATA_WIDTH	Number of bits in the input vector d	10
COUNT_WIDTH	Number of bits required to represent the count	$clog2(DATA_WIDTH+1) = 4

Why use DATA_WIDTH + 1 inside $clog2?

For DATA_WIDTH = 10, the count can range from 0 through 10. That is 11 possible values. Four bits are needed because $2^3 = 8$ is not enough, while $2^4 = 16$ can represent all values. Therefore COUNT_WIDTH = $clog2(10+1) = 4.

> **The +1 is important because the maximum count itself (DATA_WIDTH) must be representable.**

How the for-loop counts the bits

The loop visits each bit from d[0] to d[DATA_WIDTH-1]. Before counting starts, count is set to zero. Each input bit is then added to count.

Step	Bit examined	Bit value	Running count
Start	—	—	0
1	d[0]	1	1
2	d[1]	1	2

3	d[2]	1	3
4	d[3]	0	3
…	Remaining bits	…	Final count

This parameterised combinational implementation is used in the project.

```
module once_count #(
  parameter DATA_WIDTH = 10,
  parameter COUNT_WIDTH = $clog2(DATA_WIDTH+1))
(
  input  [DATA_WIDTH-1:0] d,
  output [COUNT_WIDTH-1:0] y
);

  integer i;
  reg [COUNT_WIDTH-1:0] count;

  always @(*) begin
    count = 0;
    for(i = 0; i < DATA_WIDTH; i = i+1) begin
      count = count + d[i];
    end
  end
  assign y = count;
endmodule
```

Important RTL points

- **always @(*)** describes combinational logic. The block reevaluates when any signal it reads changes.
- **count = 0** initialises the accumulator each time the combinational block executes. This prevents the previous calculation from being carried into the next one.
- **for loop** iterates over every input bit. Because the loop bound uses DATA_WIDTH, it adapts when the parameter changes.

> **This is a behavioral description of combinational addition. Synthesis tools implement the required combinational arithmetic hardware from the RTL.**

- **assign y = count** connects the calculated count to the output.

Testbench with a fixed random seed
The testbench applies ten pseudo-random input values. The integer NOFONCE is passed as the seed to $random. The system task updates the seed as random values are generated.

```
module tb;

  parameter DATA_WIDTH = 10;
  parameter COUNT_WIDTH = $clog2(DATA_WIDTH+1);

  reg  [DATA_WIDTH-1:0] d;
  wire [COUNT_WIDTH-1:0] y;

  integer NOFONCE=9999;

  once_count #(
    .DATA_WIDTH(DATA_WIDTH),
    .COUNT_WIDTH(COUNT_WIDTH)) dut (.d(d),.y(y));

  initial begin
    repeat(10) begin
      d = $random(NOFONCE);
      #1;
      $display("d=%b, y=%0d (%b)", d, y, y);
    end
    $finish;
  end

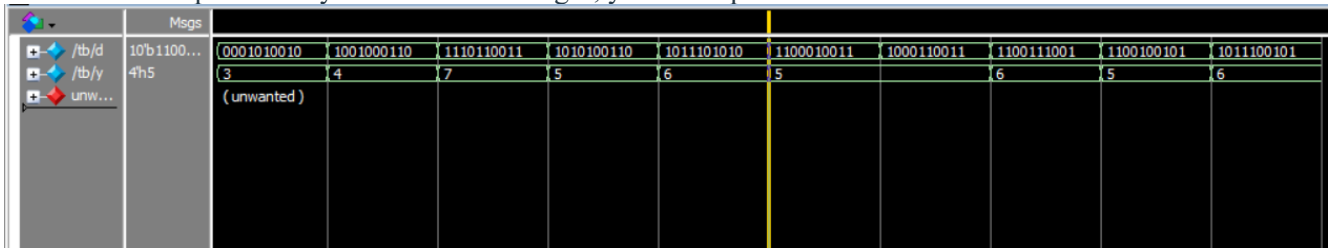
endmodule
```

Example input and output

10-bit input d	Number of 1s	4-bit output y	Decimal
0000000000	0	0000	0
0000001111	4	0100	4
0000010101	3	0011	3
1111111111	10	1010	10

Simulation and waveform

Run the RTL and testbench in a Verilog simulator such as QuestaSim or an online simulator. Observe the input vector d and output vector y. Whenever d changes, y should update to show the number of 1s in d.



Key learning outcomes

- Writing reusable RTL using parameters.
- Calculating output width with $clog2.
- Describing combinational logic using always @(*).
- Using a for-loop to process a variable-width input vector.
- Creating a testbench with pseudo-random stimulus and a fixed seed.
- Checking the relationship between binary input bits and the population-count output.

Conclusion

> **The parameterized Ones Counter counts the number of logic-1 bits in an input vector of configurable width. By using DATA_WIDTH, COUNT_WIDTH and a for-loop, the same RTL can be reused for multiple input sizes without rewriting the counting logic.**

Project link

count_once.v - EDA Playground